

PL-SQL

AM.SC.U4AIE23166

NISHANT SHAH

```
create or replace function add_two(a int , b int)
returns int as
$$
declare
    c int ;
begin
    c := a+b;
    return c;
end
$$
Language plpgsql;

select add_two(5, 7);
```

	add_two integer
1	12

Q2

```
create or replace function even_odd(a int)
returns varchar as
$$
begin
    if a % 2 = 0 then
        return 'even';
    else
        return 'odd';
    end if;
end;
$$
Language plpgsql;

select even_odd(5);
```

Showing rows: 1 to 1		Page No:
	even_odd character varying	
1	odd	

```

CREATE TABLE dept (
    dno INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE emp (
    eno INT PRIMARY KEY,
    ename VARCHAR(50),
    sal NUMERIC(10,2),
    dno INT REFERENCES dept(dno)
);

INSERT INTO dept VALUES
(10, 'Sales'),
(20, 'Finance'),
(30, 'IT');

INSERT INTO emp VALUES
(4, 'David', 48000, 10),
(5, 'Emma', 62000, 20),
(6, 'Frank', 53000, 30),
(7, 'Grace', 70000, 10),
(8, 'Henry', 45000, 20);

INSERT INTO emp VALUES
(1, 'Alice', 50000, 10),
(2, 'Bob', 60000, 20),
(3, 'Charlie', 55000, 30);

```

```

CREATE OR REPLACE FUNCTION get_sal(emp_no emp.eno%type)
RETURNS emp.sal%type AS
$$
DECLARE
    salary emp.sal%type;
BEGIN
    SELECT sal INTO salary
    FROM emp
    WHERE eno = emp_no;

    RETURN salary;
END;
$$ LANGUAGE plpgsql;

SELECT get_sal(2);

```

☰+	📄	▼	📋	▼	🗑️
	get_sal				
	numeric				
1	60000.00				

```
▼ create or replace function get_num_emp(dept_no dept.dno%type)
    returns int as
$$
    declare
        num_emp int ;
▼    begin
        select count(eno) into num_emp from emp
        where dno = dept_no;

        return num_emp;
    end;
$$
language plpgsql;
select get_num_emp(10);
```



	get_num_emp	integer	🔒
1		3	

```

CREATE OR REPLACE FUNCTION hike_sal(emp_no emp.eno%type)
RETURNS void AS
$$
DECLARE
    avg_sal_dept emp.sal%type;
    curr_sal emp.sal%type;
BEGIN
    -- Get current salary
    SELECT sal INTO curr_sal
    FROM emp
    WHERE eno = emp_no;

    -- Get department average salary
    SELECT AVG(sal) INTO avg_sal_dept
    FROM emp
    WHERE dno = (SELECT dno FROM emp WHERE eno = emp_no);

    -- If current salary < dept avg, increase by 10%
    IF curr_sal < avg_sal_dept THEN
        UPDATE emp
        SET sal = curr_sal + (curr_sal * 0.10)
        WHERE eno = emp_no;
    END IF;

END;
$$ LANGUAGE plpgsql;

SELECT hike_sal(2);
SELECT * FROM emp WHERE eno = 2;

```

hike_sal	void
1	

Data Output Messages Notifications

eno	ename	sal	dno
[PK] integer	character varying (50)	numeric (10,2)	integer
1	2 Bob	60000.00	20

```
DROP TABLE IF EXISTS EMP_PROJ;  
DROP TABLE IF EXISTS Emp;
```

```
CREATE TABLE Emp(  
    eno INT,  
    ename VARCHAR,  
    sal NUMERIC(7,2)  
);
```

```
CREATE TABLE EMP_PROJ(  
    eno INT,  
    pno INT,  
    prjt_hrs INT,  
    PRIMARY KEY(en, pno)  
);
```

```

create table Emp(eno int, ename varchar, sal numeric(7,2));
create table EMP_PROJ(eno int, pno int, prjt_hrs int,primary key(eno,pno));
--2(a)
CREATE OR REPLACE FUNCTION ProjectLoad(p_eno INT)
RETURNS NUMERIC AS $$
DECLARE
    total_hours NUMERIC := 0;
BEGIN
    SELECT COALESCE(SUM(prjt_hrs),0)
    INTO total_hours
    FROM EMP_PROJ
    WHERE eno = p_eno;

    RETURN total_hours;
END;
$$ LANGUAGE plpgsql;

```

```

/
-----

```

CREATE FUNCTION

Query returned successfully in 36 msec.

```

/
--2(c)
DO $$
DECLARE
    avg_sal NUMERIC;
BEGIN
    SELECT AVG(sal) INTO avg_sal FROM Emp;

    UPDATE Emp
    SET sal = avg_sal
    WHERE sal < avg_sal;

    RAISE NOTICE 'Salary updated';
END;
$$;

```

NOTICE: Salary updated

DO

Query returned successfully in 36 msec.

3.

```
CREATE OR REPLACE FUNCTION CheckAndDeleteItem(p_ino INT)
RETURNS TEXT AS $$
DECLARE
    tran_count INT;
BEGIN
    SELECT COUNT(*)
    INTO tran_count
    FROM Transaction
    WHERE ino = p_ino;

    IF tran_count < 2 THEN
        DELETE FROM Item
        WHERE ino = p_ino;

        RETURN 'Item deleted - less than 2 transactions';
    ELSE
        RETURN 'Item NOT deleted - at least 2 transactions exist';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

4

```
--4
CREATE OR REPLACE FUNCTION UpdateCustomerClass(p_threshold NUMERIC, p_cust_no INT)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    total_bal NUMERIC := 0;
BEGIN
    SELECT COALESCE(SUM(bal),0)
    INTO total_bal
    FROM ACCOUNTS
    WHERE cust_no = p_cust_no;

    IF total_bal > p_threshold THEN
        UPDATE CUSTOMER
        SET c_type = 'A'
        WHERE cno = p_cust_no;

        RETURN 'Customer class updated to A';
    ELSE
        UPDATE CUSTOMER
        SET c_type = 'B'
        WHERE cno = p_cust_no;

        RETURN 'Customer class updated to B';
    END IF;
END;
$$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 131 msec.

B

```

CREATE OR REPLACE FUNCTION CloseBranch(p_old INT, p_new INT)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
BEGIN
    -- Transfer all accounts
    UPDATE ACCOUNTS
    SET br_no = p_new
    WHERE br_no = p_old;

    -- Delete old branch
    DELETE FROM BRANCHES
    WHERE br_no = p_old;

    RETURN 'Branch closed and accounts transferred.';
END;
$$;

```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 77 msec.

C:

```

--C
CREATE OR REPLACE FUNCTION SafeWithdraw(p_ac_no INT, p_amount NUMERIC)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    curr_bal NUMERIC;
BEGIN
    SELECT bal INTO curr_bal
    FROM ACCOUNTS
    WHERE ac_no = p_ac_no;

    IF curr_bal >= p_amount THEN
        UPDATE ACCOUNTS
        SET bal = bal - p_amount
        WHERE ac_no = p_ac_no;

        RETURN 'Withdrawal successful.';
    ELSE
        RETURN 'Insufficient funds. Withdrawal denied.';
    END IF;
END;
$$;

```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 84 msec.